Reverse Engineering of the MTTL-W01 Smart Power Strip's Architecture and Protocols

A Comparison of Official LG U+ Cloud Control and Local Control Methods

By Eng. Mohamed Eid Mohamed

Applied Research Thesis on IoT Security and Embedded Software Reverse Engineering

Device: Tonly Technology / LG U+ ecosystem — Model MTTL-W01 (MULTITAP)

Processor: Realtek RTL8711AF (Ameba1, ARM Cortex-M3)

Research Period: May 2026 – October 2026

Version Completed: October 15, 2026

Research and Ethical Statement. *All tests were done on devices I personally own. This paper does not provide instructions for attacking other systems, stealing accounts, or breaking service protections. Identifiers for any third-party cloud services used in experiments have been removed and replaced with general descriptions. The main comparison is between the official LG U+ method and local control methods (like server emulation, standalone firmware, and a local text protocol). Artificial intelligence tools were used only for language polishing and formatting assistance; the research design, experiments, analysis, and conclusions are entirely the author's own work.*

Abstract

This thesis systematically documents the architecture and protocols of the MTTL-W01 smart power strip. It compares the trust models of the official LG U+ cloud control versus available local control options. The research involved capturing live network traffic, disassembling firmware, setting up experimental local servers, and comparing static analysis findings with later released source code.

The device works in several separate layers:

1. Local setup: Uses SoftAP and a private TCP protocol on port 30300.
2. Cloud connection: Registers and runs using MEF/HTTP and MQTT over TLS within the oneM2M framework.
3. Over-the-air updates (OTA): Includes checks for new versions, XOR-scrambled images, integrity checks, and boot-path signature verification.
4. Hardware outlets: Relays are not controlled directly by GPIO. Instead, a two-wire bus (Clock/Data on PB2/PB3) connects to an external driver circuit at address 0xA8, register 0x13. This was confirmed by local-control source code released in October 2026.

Main finding: On the cloud path, security, permissions, and subscription checks are mostly handled by the server; no strong per-command encrypted subscription check was observed in the static analysis of the firmware. Local paths shift trust to your home network, the firmware's integrity, and local key management. These local options have trade-offs regarding encryption, authentication, scalability, and reliability during internet outages.

Keywords: reverse engineering, Internet of Things, oneM2M, MQTT, SoftAP, OTA, Realtek Ameba1, local control, embedded device security, LG U+.

Data and Code Availability

The SoftAP local provisioning script developed and used in this research is provided together with this thesis for educational and verification purposes, strictly on devices owned by the researcher. The original stock firmware image (comMTTL-

W01_1.0.66.fwr), standalone firmware sources, local-server implementations, and the up: text-protocol bridges are publicly available in the community GitHub repositories listed in the References section.
Vendor disclosure. No vendor report was filed regarding the SoftAP provisioning findings because the device is obsolete and no longer available from LG U+.

1. Introduction and Research Problem

Smart power strips are now key parts of connected homes. They control power to important devices (like heating, network gear, security lights, pumps), report energy usage, and are linked to user accounts and subscription plans. However, their internal communication methods are rarely published, and users typically only interact with them through a smartphone app.

1.1 Main Research Goal

The main goal of this thesis is to thoroughly and verifiably document the architecture and protocols of the MTTL-W01 smart power distribution device. Then, it compares the trust models of the official LG U+ cloud control against available local control methods (server emulation, standalone firmware, local text protocol).

This research isn't about breaking security or enabling attacks. Instead, it aims to:

1. Help researchers and advanced users understand the security, functional, and operational trade-offs between relying on the cloud versus operating locally.
2. Document the limitations of static analysis when compared to later source code releases or hardware measurements.
3. Provide a way to separate successful communication at the transport level (like an MQTT PUBACK) from actual hardware actions.
4. Contribute to the study of responsible reverse engineering of consumer IoT devices.

The MTTL-W01 is a good example because it uses an older embedded Wi-Fi chip (RTL8711AF, Ameba1 family), has a closed local setup protocol (SoftAP + TCP 30300), connects via oneM2M/MQTT which is common in the Korean LG U+ network, and later had community local control options with public source code available (October 2026).

2. Research Questions and Hypotheses

2.1 Main Research Questions

- Q1. What happens step-by-step when the device is reset to factory settings?
- Q2. How does the ongoing communication channel work after the device connects to the home network using the official LG U+ path?
- Q3. Is getting confirmation that a message was sent (like MQTT PUBACK) enough to prove that a command was actually carried out by the relay?
- Q4. What are the limits of analyzing the Image2 binary compared to what later source code or the relay hardware shows?
- Q5. How do the trust and permission systems differ between official cloud control and local control?

2.2 Working Hypotheses

- H1. The device clearly separates three functions: (a) network setup, (b) identity registration, and (c) the command/event channel.
- H2. Most rules for subscriptions and application-level permissions are managed on the server, not within the device firmware.
- H3. The GPIO pins seen in disassembly might not directly control relays one by one, but instead communicate through a main bus.

- H4. Local control is possible in different ways: by mimicking the operator's path, by replacing the firmware, or by using an alternative text protocol on the original firmware.

3. Related Literature and Platform Background

3.1 Security of Smart Plugs and Power Strips

Many security studies on consumer smart plugs (including devices from Tuya, TP-Link, and Meross) have found that they often use undocumented protocols, and the local attack surface (like SoftAP or management ports) is usually less secure than the cloud connection. Other research has highlighted the privacy risks of constant energy monitoring, which can reveal when people are home.

3.2 oneM2M Standard and MQTT Security

The oneM2M standard provides a detailed model for resources (CSE, AE, ACP, CIN, subscriptions) and security features for identification, authorization, and managing certificates. In practice, data is often sent using MQTT or HTTP. Research on MQTT security points out the risks of exposed brokers and weak identity checks. For a closed device, the main risks involve client secrets, certificate management, the integrity of the registration process, and the potential reuse of stolen credentials.

3.3 Realtek Ameba1 Family

Official Realtek documents describe the Image1 (bootloader) / Image2 (application) structure, how partitions are switched, image-region signatures, and the OTA address in system data. Understanding this is key to realizing why analyzing only Image2 doesn't reveal the full boot verification process. The RTL8711AF uses an ARM Cortex-M3 processor at moderate speeds, which helps explain why features requiring more memory were dropped in standalone local versions.

3.4 Community Work on MTTL-W01

Later, community efforts documented local control methods: a local server that mimics operator endpoints using router redirection (DNAT); standalone firmware with a web interface and MQTT discovery for local networks; and bridges/local text protocols (like the up: message family). These works are used here purely for comparison.

4. Device Under Study: Specifications and Identity

Field	Value
Apparent manufacturer	Tonly Technology / LG U+ ecosystem
Model	MTTL-W01
Type	MULTITAP — four independent outlets + measurements
SoC	RTL8711AF (Ameba1)
Radio	2.4 GHz only (observed provisioning behaviour)
Reference stock firmware	comMTTL-W01_1.0.66.fwr — 327944 bytes
SHA-256 of reference	d780b578af69d52f3a05191a8e7d91a20e05085a912722327481cd5663682c04

After removing XOR obfuscation from the 1.0.66 firmware image, confirmed text strings include MTTL-W01, MULTITAP, LGAPMODE0010, TONLY_TAP, pool.ntp.org, references to oneM2M/U+, and LGU+-TAP-HW002. Notably, the image does not contain any domain names belonging to third-party experimental cloud services.

5. Methodology, Tools, and Detailed Timeline

5.1 Methodological Principles

This work follows three core principles: clearly separating what is proven by live network captures or tests, what is inferred from disassembly, and what is taken from later published source code; never assuming that successful

communication at the transport level proves success at the application or hardware level; and documenting failures with the same level of detail as successes.

5.2 Tools and Techniques

Tool / Technique	Usage
Network capture (Wireshark)	SoftAP, TLS, MQTT
Frida + Android emulation	Interception of mobile-app paths
mitmproxy / controlled TLS proxy	Study of REST from the application side
Ghidra	Image2 disassembly, function tracing, string analysis
Python (experimental servers)	MEF/HTTP, MQTT, OTA, control tooling
OpenSSL	Certificate inspection and PEM chain handling
XOR de-obfuscation of FWR images	String extraction and family verification

5.3 Detailed Work Timeline

Period	Main activity
May–June 2026	Laboratory setup, SoftAP, first captures
July 2026	REST from the application side, state/energy/schedule paths
August 2026	Local servers, OTA, oneM2M registration; first analytical closure
September 2026	Deepening of command gateway, certificates, multi-device paths
October 2026	Community local paths; standalone source; relay-bus addendum; this thesis

All dates refer to completed work. This version was finalized on October 15, 2026.

6. Overall System Architecture and Control Paths

6.1 Official Cloud Control Path (LG U+)

In the official setup, the mobile app communicates with LG U+ central servers using HTTPS REST for account, status, schedules, and energy data. The device itself maintains a oneM2M connection over MQTT/TLS (and uses MEF interfaces for registration, certificates, and OTA updates). The device acts as a client; the server handles identity verification, stores policies, and typically interprets permissions linked to the user's account. After initial setup, the mobile app does not communicate directly with the device over the local network for control commands.

Flow: User App (HTTPS REST) → LG U+ Central Cloud (oneM2M / MQTT/TLS + MEF) → MTTL-W01 on Home Network → Outlet Driver Circuit → Connected Devices.

6.2 Experimental Third-Party Cloud Intermediary

During the project, a system from a third party that provided an application/cloud layer for similar devices was examined. Much of the registration process could be duplicated locally. Confirmation messages at the MQTT transport level were frequently observed. However, proof that a reconstructed control command actually changed the relay state on that specific system could not be obtained with the tools available at the time.

6.3 Local Control Paths

- A. Server Emulation: Uses the original or slightly modified firmware, communicates with a local MEF and MQTT server, and requires router redirection (DNAT).
- B. Standalone Firmware: Provides a web interface on the local network, uses MQTT Discovery, and does not connect to the operator's cloud.
- C. Text Protocol (up:): A simple TCP text protocol (commands such as up:onoff and up:getinfo) that is present and functional in the original stock firmware; requires only a minimal local server.

These three local approaches differ significantly in their trust model, operational complexity, and how complete their features are.

7. Local Provisioning Protocol (SoftAP)

The local setup phase is the only time the device offers a temporary Wi-Fi access point and accepts configuration via a dedicated TCP port. Understanding this phase is crucial for lab testing and for evaluating the local security risks.

7.1 Network Observed After Reset

After a proper factory reset, the device enters setup mode and creates a SoftAP network. The network name (SSID) follows the pattern TONLY_TAP_ followed by a code derived from the device's MAC address. The password is LGU_ plus the same code. The device itself acts as the gateway for this temporary network, usually at address 192.168.1.1. Configuration data is sent only over TCP port 30300. This means anyone within radio range who can guess or calculate the MAC-derived password can complete the setup. Once setup is done, the device exits SoftAP mode and connects to the specified home network.

7.2 Message Header and Command Format

All messages seen on the setup port start with the standard text header LGAPMODE0010. After the header comes a single command byte and a variable-length data section. Command 0x67 is used by the setup tool to request device information (model, firmware version, MAC address). Command 0x65 indicates the length of the following data block, which contains the Wi-Fi network name (SSID) and password. The data section observed in captured traffic has a 32-byte field for the SSID and a 32-byte field for the password, with the rest padded with zeros. The protocol is binary and undocumented. However, the fixed header and the few observed commands were enough to recreate the full setup process in the lab using a custom Python tool.

7.3 Local Attack Surface and Security Implications

Since the SoftAP password is automatically generated from the MAC address, the security of the setup phase depends on physical or radio proximity rather than having a secret code separate from the hardware identifier. Once setup is complete, the SoftAP disappears, and further management is done either through the cloud or whatever options the firmware provides on the home network. The SoftAP vulnerability is temporary but significant: any attacker who can get a radio signal close enough during the short window after a reset can connect the device to a network they control.

8. Cloud Registration and Runtime Path (MEF / oneM2M / MQTT)

After the device connects to the home network, it goes through a multi-step registration process that ends with a continuous MQTT connection carrying oneM2M data.

8.1 Layer Overview

The ongoing connection involves three interacting layers. First, a set of HTTP interfaces (called MEF-style interfaces) handle initial registration, getting or renewing certificates, and checking for OTA updates. Second, after successful registration, the device maintains a long-term MQTT connection over TLS, typically on port 18831. Third, within the MQTT connection, the device creates and manages oneM2M resources—specifically an Application Entity (AE), Access Control Policies (ACP), and Content Instances (CIN)—and subscribes to command topics. Commands and status updates are then sent as MQTT PUBLISH messages with payloads formatted according to oneM2M resource structures. A successful MQTT connection alone doesn't guarantee the device is fully registered or will accept any command; the oneM2M resource structure and ACP rules must also be correctly configured.

8.2 Relevant oneM2M Elements and Permission Placement

In the official LG U+ system, most decisions about Access Control Policies are made on the server. The device creates an AE identified by its hardware ID, but it doesn't seem to check for a per-command "is the requesting user subscribed?" check inside the firmware. Instead, it relies on the cloud to only send commands that have already passed account-level and subscription-level authorization. Therefore, an attacker who can send a properly formatted MQTT message to the correct topic after registration might still be blocked by server-side filters. Conversely, an attacker who completely replaces the cloud takes on the responsibility of implementing any desired authorization logic.

8.3 MQTT over TLS Channel

All observed MQTT traffic used TLS 1.2. Session credentials (device IDs, tokens, and related key material) were obtained dynamically from the earlier MEF HTTP exchange, not hard-coded in the firmware. A PUBACK message from the broker only confirms that the MQTT client software accepted the message; it doesn't guarantee that the application software understood the data or that any hardware action took place. This distinction was critical during testing: many recreated commands received a PUBACK but resulted in no visible change in the power outlet's state.

8.4 Experimental Intermediary Path (Limited Notes)

When an experimental third-party cloud system was studied, large parts of the MEF registration process and the subsequent MQTT connection could be replicated on a local server. Extracting dynamic credentials from MEF headers, presenting certificates, and subscribing to topics all worked. However, definitive proof that a custom control command triggered a physical relay change on that specific system was not achieved within the scope of the project. This experience reinforced the methodological rule that success at the transport level should never be treated as proof of success at the hardware level.

9. Over-the-Air Updates (OTA) and Firmware Image Structure

The OTA process and the internal structure of the firmware image together determine how easy it is to modify the device locally and how well it resists unauthorized code execution.

9.1 Image1 / Image2 Model on Ameba1

Following the Realtek Ameba1 architecture, the flash memory is divided into at least two main image areas. Image1 contains the bootloader: it checks the integrity and signature of Image2, selects the active partition if dual-bank updates are supported, and then transfers control to the starting point of Image2. Image1 does not process application commands and does not provide a handler table. Therefore, any static analysis focused only on Image2 can detail the application logic but won't reveal the exact policy enforced during boot. This separation explains why modified Image2 files that passed internal checks were still rejected by the original devices: the crucial verification happens earlier in the boot sequence.

9.2 XOR Obfuscation

Original and related firmware images are distributed in an obfuscated form. A 16-byte key is taken from byte 16 of the file and applied repeatedly using XOR to the rest of the image. The same key family was used to successfully de-obfuscate both the reference 1.0.66 image and earlier experimental images, showing that the obfuscation method has remained consistent across the versions examined. After de-obfuscation, common text strings (MTTL-W01, LGAPMODE0010, TONLY_TAP, oneM2M identifiers) become visible.

9.3 Practical OTA Mechanism

The observed OTA process starts with comparing version strings. In some test setups, the device was sensitive to the character used as a separator in the version identifier (e.g., . vs _), requiring the advertised version to be adjusted before the download would begin. Once the version check passes, the device downloads the (still obfuscated) image, verifies a checksum in the image header or footer, and then relies on the bootloader for final acceptance. Successful OTA installation of custom and community-prepared firmware images was achieved when

the original port numbers and endpoints expected by the stock firmware were preserved. Failures observed in some third-party or experimental setups were associated with altered port numbers (for example changes to the MEF/OTA or controller ports). A correct checksum remains necessary, and the boot-path checks (most likely performed by Image1) still apply; images that satisfy those checks install normally through the documented OTA paths and community recovery tools.

9.4 Stock Image 1.0.66 Reference

File: comMTTL-W01_1.0.66.fwr Size: 327944 bytes

SHA-256: d780b578af69d52f3a05191a8e7d91a20e05085a912722327481cd5663682c04

Header 16 bytes: f400050000600010ffffffffffffffff

XOR key: 3f5a27e8d8fb85f9bd4d51196c4f5159

10. Command Path from Application Layer to Hardware (Relays)

Tracing a control command from the network all the way to the mechanical switch activation was the most revealing—and most often misunderstood—part of the analysis.

10.1 What Static Analysis Established (August 2026)

Disassembling the Image2 firmware revealed a clear path from the command interpreter to a GPIO write instruction: POWER parser → shared state → tonly_bsp_tap_task → gpio_set (read-modify-write) → STR. The parser recognizes a power command, updates a shared data structure, and then schedules or directly calls a board-support-package task. This task eventually performs a read-modify-write operation on a GPIO register and executes a store (STR) instruction. At the time this path was found, it was tempting to assume that the STR instruction was the final step that powered a relay. Later evidence showed this interpretation was incomplete.

10.2 What Published Source Clarified (October 2026)

In October 2026, the source code for an experimental local-control firmware was published. Under a specific compile-time setting (MTTL_RELAY_TEST / opt-in), the code shows that the GPIO pins accessed by the STR instruction are not wired as four separate relay drivers. Instead, they form a two-wire bit-banged bus: Clock pin = PB2 (0x12), Data pin = PB3 (0x13), target device address = 0xA8, output register = 0x13, frame length = exactly four bytes. The protocol follows a standard start/send/receive/stop sequence with acknowledgments, similar to a software implementation of I²C. The four-byte frame contains bits for both the outlet switches and the indicator LEDs. After a frame is sent to activate a coil, the code waits about 100 ms and then sends a frame to release it (return to neutral). Importantly, comments in the source code state that the verified flag returned by the hardware-port layer indicates only that the bus interface is ready, not that a physical switch has closed.

10.3 Fair Limits of Inference

Claim	Status
Published standalone source contains output bus PB2/PB3 → 0xA8 / reg 0x13	Proven from published code
Output frame is 4 bytes containing outlet/LED bits	Proven from published code
This path is enabled by default in all production builds	Not proven — guarded by compile-time flags
Identical bit maps exist in stock 1.0.66 Image2	Not proven — requires binary or live comparison
“verified” equals mechanical contact confirmation	Explicitly rejected by source comments

10.4 Updated End-to-End Control Path

Combining the earlier static analysis with the later source code information reveals the complete path: Application / oneM2M / MQTT command → Command parser in Image2 → Shared state → BSP task → GPIO write (STR) → GPIO PB2/PB3 acting as clock/data → Bit-bang protocol to address 0xA8 → Register 0x13 receives a 4-byte frame → Coil-activation frame followed by a release frame → External driver circuit controls the mechanical relays.

The key takeaway from a methodological standpoint is that the last instruction seen in disassembled code might only be the first step in a process involving communication outside the main chip. Assuming this instruction is the final hardware action leads to incorrect assumptions about security risks and what a reconstructed network command can achieve.

11. Security: Authentication, Permissions, Subscription, Certificates, Signing

The security features of the MTTL-W01 are spread across several layers and cannot be simply labeled as secure or insecure.

11.1 Authentication Model on the Cloud Path

Authentication is two-part. For the user, the mobile app authenticates to LG U+ services with an account password and then uses short-lived access tokens that are refreshed by a longer-lasting refresh token. For the device, the firmware presents a device certificate during the MEF registration process and receives dynamic session credentials. These credentials are then used to establish the MQTT/TLS connection. The two authentication domains—user account and device identity—are linked only within the operator's systems; the device itself never sees the user's password.

11.2 Permissions and Subscription

Subscription status and most application-level authorization decisions are handled by the server. The firmware does not include a strong cryptographic subscription check that would require, for example, a per-command token generated from a user-specific secret. Consequently, once an attacker can get the device to accept a message on the correct MQTT topic, the remaining obstacles are those set by the cloud (or by whatever local server has replaced the cloud). This setup simplifies the device firmware but makes the server the sole enforcer of commercial subscription policies.

11.3 Certificates and TLS

All runtime traffic examined used TLS 1.2. Practical challenges when building local emulators involved completing the certificate chain, correctly aligning validity periods (notBefore / notAfter), and properly presenting the PEM data. These issues are related to configuration and trust anchors, not flaws in the underlying encryption. Once a consistent local Public Key Infrastructure (PKI) was set up, the device completed the TLS handshake and proceeded to the MQTT CONNECT stage.

11.4 Update Image Signing

Official update images and properly prepared custom/community images (standalone and local-patched) were successfully installed via OTA when the original port numbers and service endpoints expected by the stock firmware were left unchanged. In contrast, experimental setups that altered those ports (for example the MEF/OTA check port or the local controller port used by some recovery tools) produced consistent OTA failures. A visible checksum check is necessary but not sufficient; a further boot-path verification step (most likely inside Image1) still applies. This paper does not claim to have reverse-engineered the exact verification mechanism. The practical observation is that preserving the original port configuration allows successful OTA of custom firmware, while changing those ports from outside is a common cause of failure.

11.5 SoftAP as Local Attack Surface

As mentioned in Section 7, the SoftAP password is derived from the MAC address. The period of exposure is limited to the time after a reset and before successful setup, but within that window, physical proximity is enough. After setup, the SoftAP disappears, and any remaining local access points are those intentionally exposed by the chosen firmware.

11.6 Trust-Model Comparison

Dimension	LG U+ cloud	Local control

Primary trust anchor	Operator account + server policies	Home network + firmware integrity + local keys
Permission enforcement	Predominantly server-side	Local policy or none
Subscription gate	Server-centric	Absent or application-level
Update trust	Boot-path signature	Source of the image + local decision
Continuity without internet	Limited	Full (for pure local paths)

This table shows that security is not a simple measure. Each method prioritizes a different balance of convenience, centralized policy enforcement, and offline reliability.

12. Published Local Control Paths and Methodological Comparison

12.1 Local Server Emulating Operator Endpoints

This emulation approach uses the original (or nearly original) firmware but redirects the device's network destinations to a local server. This server provides certificates, registration responses, and an MQTT broker. It typically requires setting up certificates specific to the installation, understanding MEF and MQTT flows, and often configuring router-level network redirection (DNAT). The main advantage is that it behaves like the official system; managing multiple devices from one interface is also possible. The main drawbacks are operational complexity and the residual security risk if any management interface is accidentally exposed outside the local network.

12.2 Standalone Firmware

Standalone firmware replaces the cloud connection logic with a local network web dashboard and an MQTT Discovery feature for systems like Home Assistant. Newer versions also allow firmware updates through the device's own web interface. This approach offers complete independence from the operator's services and benefits from publicly available source code for certain functions. Trade-offs include the need to trust the origin of the firmware image, the lack of strong web authentication in the published design (only safe on a trusted local network), and limitations in memory that prevent advanced features like multi-device grouping within the device itself.

12.3 Local Text Protocol up:

A simple TCP text protocol using commands such as up:onoff and up:getinfo is present and functional in the original stock firmware. This has been confirmed by community implementations (notably the MatterBridge project) that operate against unmodified 1.0.66 devices. Its advantages are simplicity and the ability to work with the original firmware without modification. Its limitations include the lack of encryption for data sent over the local network and dependence on the protocol remaining consistent across firmware updates.

12.4 Focused Comparison Table

Dimension	LG U+ cloud	Server emulation	Standalone	up: protocol
External account	Yes	No	No	No
Permissions location	Operator server	Home policy	Home policy	Essentially none
Setup complexity	Low (end user)	High	Medium	Low–medium
Modified image needed	No	Sometimes	Yes	No
Main threat surface	Account / cloud	LAN keys + certs	FWR source + LAN	Open LAN

13. Detailed Experimental Results and Limitations

13.1 What Is Considered Proven

- Characterization of SoftAP and the LGAPMODE0010 protocol through live setup tests.
- Distinction between the user-facing REST API and the device's communication channel.

- Characterization of oneM2M/MQTT registration components and topic subscription behavior.
- The difference between receiving a PUBACK and actual hardware activation.
- The structure of the FWR image, its XOR obfuscation, and the size/hash of the 1.0.66 reference file.
- The limitations of analyzing Image2 compared to the boot process policy.
- Interpretation of the relay bus based on published local-path source code.
- The server-centric nature of subscription and application permissions.

13.2 What Was Not Proven

- Confirmed relay activation from a fully reconstructed MQTT command on the experimental third-party system.
- The exact mapping of bits to outlets within the 4-byte frame for the official original firmware without live measurements.
- Breaking the boot signature system—this was beyond the scope of the tools used.

13.3 Methodological Limitations

Static analysis without complete manufacturing symbols can lead to errors in naming functions. Selective network capture might miss infrequent communication paths. Later source code is not identical byte-for-byte to earlier binary files. Findings for the RTL8711AF processor may not apply to all LG U+ IoT devices.

14. In-depth Discussion

14.1 Why the Cloud Path Took Longer Than Local Paths

The operator's path is designed to be a black box for the end user. Replicating its communication required rebuilding trust in TLS, understanding device identity, the oneM2M resource structure, and the exact command format. Community local paths simplified the goal to controlling the outlets and reading energy with the fewest intermediary steps.

14.2 Security Is Not a Single Property

A device might be resistant to fake updates due to a secure boot process, yet vulnerable to an attacker within SoftAP range, and simultaneously rely on the cloud for permission decisions. An accurate assessment must consider these layers together.

14.3 Privacy and Energy Telemetry

Energy usage data sent to the cloud can reveal lifestyle habits. Local control reduces this data exposure but shifts the responsibility for updates and backups to the user.

14.4 The Engineering Lesson from GPIO

Reaching a GPIO store instruction (STR) was an analytical success; assuming it was the end of the activation process was premature. The hardware involved an additional communication protocol. The general lesson is that the last instruction visible in disassembled code might just be the start of an external bus transaction.

14.5 Memory Constraints and Design Impact

The rejection of advanced multi-device features in the device's web interface is consistent with the limited memory of the RTL8711AF. Grouping features at a higher level (e.g., within Home Assistant) remains the practical solution.

14.6 Service Continuity After Full Power Loss

Community standalone versions addressed reconnection after complete power outages. Official cloud paths rely on re-registration and server-side session status, which adds extra delay during recovery.

15. Conclusions and Expanded Security & Operational Recommendations

15.1 Main Conclusions

- The MTTL-W01 is a complex system involving SoftAP, MEF, oneM2M/MQTT, OTA, and a hardware bus.
- Official cloud control centralizes policies, permissions, and subscription management on the server.
- Confirmation of message delivery (PUBACK) does not guarantee hardware action.
- Local control is a well-developed community option offering various levels of accuracy and complexity.
- The two-wire bus (PB2/PB3 to 0xA8/0x13) helps explain why the outlet mapping was initially hard to determine through static analysis alone.
- Verification of the boot-path signature limits the risk of arbitrary code execution over the network via fake OTA updates.
- On-chip memory limitations influence which features can be implemented in alternative local firmwares.

15.2 Expanded Security and Operational Recommendations

A. For Home Users / Smart Home Operators

Recommendation	Rationale
Isolate IoT devices on a separate VLAN	Limits lateral movement after a local compromise
Do not expose management ports to the internet	Local paths are designed for a trusted LAN
Treat factory reset as a security event	Returns the device to a MAC-derived SoftAP password
Prefer standalone + MQTT Discovery via a trusted broker	Currently the most operationally proven cloud-free path
Implement multi-device grouping at a higher layer	On-device memory limits reject advanced grouping
Prefer local energy monitoring when privacy matters	Cloud telemetry can reveal occupancy patterns

B. For Reverse Engineering Researchers

Recommendation	Rationale
Separate evaluation layers: radio → TLS → app → hardware	Prevents equating PUBACK with actuation
Document failures with equal rigour	Avoids result inflation
Caveat any mixing of later source with older binaries	Source is compiled under specific flags
Use only owned devices and state that fact	Ethical and legal foundation
When publishing, remove third-party infrastructure IDs	Reduces misuse potential

C. Concise Threat Model

Surface	Approx. risk	Notes
SoftAP during provisioning	Medium–high in range	MAC-derived password
MQTT/TLS after registration	Depends on certs/keys	Server-centric on official path
Forged OTA	Low–medium vs stock	Boot signature limits arbitrary code
Local web / text interfaces	High if exposed beyond LAN	Designed for trusted network
Energy telemetry	Medium (privacy)	Reveals household patterns

D. Constructive Notes for Device Designers

- Clearly separate device identity from user identity.
- Consider offering an official, documented mode for local operation.
- Use modern connection methods instead of SoftAP passwords derived from MAC addresses.
- Factor in memory limits when designing firmware for multi-device logic.

15.3 Recommendations for Future Research

- Capture an entire official control command sequence with synchronized relay measurements.

- Analyze the PB2/PB3 bus behavior on the original 1.0.66 firmware.
- Conduct a detailed binary comparison between original firmware and local-path images.
- Examine SoftAP behavior across a wider range of devices.
- Investigate ways to improve authentication for standalone firmware within memory constraints.
- Study how connections are restored after complete power loss across different control paths.

16. Appendices

Appendix A — Stock Image Reference

File: comMTTL-W01_1.0.66.fwr Size: 327944 bytes

SHA-256: d780b578af69d52f3a05191a8e7d91a20e05085a912722327481cd5663682c04

XOR key: 3f5a27e8d8fb85f9bd4d51196c4f5159

Appendix B — Glossary

Term	Meaning in this thesis
SoftAP	Temporary access point used for local provisioning
MEF	Registration / certificate / update-check interfaces on the device path
oneM2M	IoT resource framework (CSE / AE / ACP / CIN / subscription)
Image1 / Image2	Bootloader / application image on Ameba1
PUBACK	MQTT acknowledgement of a PUBLISH message
standalone	Alternative firmware that runs a local dashboard without the operator cloud
bit-bang	Software emulation of a wire protocol via GPIO
ACP / AE	Access Control Policy / Application Entity in oneM2M

Appendix C — References

1. Realtek Semiconductor Corp., “Ameba-1 (RTL8710/8711) Application Note – OTA Update,” Realtek documentation package, accessed 2026.

2. oneM2M Partnership Project, “oneM2M Technical Specification TS-0003: Security Solutions,” latest publicly available release, oneM2M.org.

3. oneM2M Partnership Project, “oneM2M Technical Specification TS-0001: Functional Architecture,” latest publicly available release, oneM2M.org.

4. S. Andy, B. R. Chandavarkar, “MQTT Security: A Comprehensive Survey of Vulnerabilities and Mitigation Strategies,” related survey literature on MQTT broker exposure, 2020–2024.

5. Independent security analyses of consumer smart-plug platforms (Tuya, TP-Link Kasa, Meross), various authors, 2019–2025 (works presented at WOOT, USENIX Security workshops, and independent researcher publications).

6. af950833, “MTTL-W01 Local Server” and “MTTL-W01 Standalone Firmware,” GitHub repositories (https://github.com/af950833/mttl_w01 and https://github.com/af950833/mttl_w01_standalone), 2026.

7. af950833, MTTL-W01 Standalone source (sdk_relay.c, meter_bus / hardware port layer), GitHub (https://github.com/af950833/mttl_w01_standalone), October 2026.

8. Author materials released with this thesis: SoftAP local provisioning script (local_provision). Original stock firmware 1.0.66 and related community sources are available in the repositories cited above.

9. ttaengz / brotherswife, “MTTL-W01 MatterBridge” (up: text protocol implementation operating on stock firmware), GitHub (https://github.com/ttaengz/mttl-w01-matterbridge and forks), 2026.

This thesis presents a multi-layered view of a home device that seems simple when controlled from a phone. It shows that comparing cloud control (LG U+) with local control is really a comparison of different trust models, not just about whether an on/off button exists. Paths that were closed off were so due to tool limitations and a

strict evidence requirement: physical control is not claimed without physical proof, and later source code is not combined with older binary code without acknowledging the methodological caveats.

End of thesis.

Author: Eng. Mohamed Eid Mohamed | Version completed: October 15, 2026 | Language: English